Zotero Organizer — User Manual

Version: 2.0
Repository: https://github.com/aitgcodes/zotero-organizer

Table of contents

---

1. What this tool does

Zotero Organizer takes a folder of PDF files and does three things automatically:

1. Extracts DOIs from each PDF (via metadata, page text, and Semantic Scholar).
2. Creates a matching collection hierarchy in your Zotero library.
3. Uploads the PDFs to a Google Drive folder and attaches the Drive links to the Zotero items.

The result is a Zotero library where every item has a properly resolved metadata record, is filed under the right collection, and links back to a cloud copy of the PDF — all without touching your original files.

No Claude or AI account is required. Claude Desktop integration is available as an optional enhancement for taxonomy suggestions (see Section 9).

---

2. Platform support

Platform	Support
macOS	Fully supported — follow this guide directly
Linux	Fully supported — follow this guide directly
Windows (WSL2)	Supported — run all commands inside WSL2
Windows (Docker)	Supported — mount your PDF folder as a volume
Windows (native)	Not supported

Why not native Windows? rclone's Google Drive authentication relies on a browser OAuth flow that is best handled in a Unix environment. The shell subprocess calls in the pipeline also assume Unix path conventions.

WSL2 is the recommended path for Windows users. Install it from the Microsoft documentation, then follow this guide from inside the WSL2 terminal. Your Windows files are accessible under `/mnt/c/` inside WSL2.

---

3. Before you begin

Two external accounts are required: Zotero (for the reference library) and Google (for Drive storage). Set them up and collect the credentials listed below before proceeding to installation.

3.1 Set up Zotero

Zotero is a free, open-source reference manager.

1. Go to zotero.org and create a free account if you do not have one.
2. Log in and navigate to **Settings → Feeds/API** (in the top-right account menu).
3. At the top of the page, note your **User ID** — it is a plain number, e.g. `1234567`.
4. Scroll to **API Keys** and click **Create new private key**.
5. Give the key a name (e.g. `zotero-organizer`), tick **Allow library access** with **Read/Write** permissions, and click **Save Key**.
6. Copy the displayed key immediately — it will not be shown again.

You now have two values to keep: - **Zotero User ID** (numeric) - **Zotero API Key** (long alphanumeric string)

3.2 Set up Google Drive

A personal Google account (`@gmail.com`) or an institutional Google Workspace account (e.g. a university `@institution.edu` account) both work.

1. Open Google Drive in your browser.

2. Navigate to or create the folder where your PDFs should be uploaded. Give it a descriptive name (e.g. `Research PDFs`).
3. Open the folder and copy the **folder ID** from the browser URL:

```
https://drive.google.com/drive/folders/1aBcDeFgHiJkLmNoPqRsTuVwXyZ
                                       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
                                       this part is the folder ID
```

You now have one value to keep: - **Google Drive Folder ID** (long alphanumeric string)

3.3 Store your credentials safely

You will enter these three values into the setup wizard in Section 4.4. Store them somewhere you can refer to:

Credential	How to retrieve if lost
Zotero User ID	zotero.org → Settings → Feeds/API
Zotero API Key	Cannot be retrieved — delete and create a new key
Google Drive Folder ID	Open the Drive folder and read from the URL

A password manager (1Password, Bitwarden, etc.) or a private local text file are both fine. Do not commit these values to a git repository.

---

4. Installation

4.1 Python

Python 3.10 or later is required. Check your version:

```
python --version
```

If Python is not installed, download it from python.org or install via your system package manager (`brew install python` on macOS, `sudo apt install python3` on Ubuntu/Debian).

Install the project dependencies. Choose one option:

Option A — conda (recommended)

The conda environment includes all CLI dependencies plus the `mcp` package needed for the optional Claude Desktop integration. It is version-pinned for reproducibility.

```
conda env create -f environment.yml
conda activate claudotero
```

If conda is not installed, download Miniconda.

Option B — pip

Installs only the four CLI dependencies. Works with a plain virtual environment or any existing Python installation. The optional MCP server will not be available.

```
pip install -r requirements.txt
```

4.2 rclone

rclone is the command-line tool that uploads files to Google Drive. It must be installed and authenticated before the setup wizard can validate your Drive connection.

Install rclone:

```
# macOS
brew install rclone

# Debian / Ubuntu
sudo apt install rclone

# Other Linux (official installer)
curl https://rclone.org/install.sh | sudo bash
```

Create a Google Drive remote:

Run the interactive configuration wizard:

```
rclone config
```

Step through the prompts as follows:

Prompt	Response
No remotes found — make a new one?	`n` (new remote)
Name	`gdrive` (or any name you prefer)
Storage type	`drive`
Google Application Client Id	Press Enter (leave blank)
Google Application Client Secret	Press Enter (leave blank)
Scope	`1` (full access to all files)
Service account credentials	Press Enter (leave blank)
Edit advanced config	`n`
Use auto config	`y`

After you type `y` for auto config, rclone opens a browser window. Sign in with the Google account that owns the Drive folder you created in Section 3.2, and click **Allow**.

Back in the terminal, confirm the remote and quit the config wizard.

Test the connection:

```
rclone ls gdrive: --drive-root-folder-id=<YOUR_FOLDER_ID>
```

Replace `<YOUR_FOLDER_ID>` with the folder ID from Section 3.2. An empty result with no error message confirms that rclone can reach the folder. (Empty output is expected for a new, empty folder.)

4.3 Clone the repository

```
git clone https://github.com/aitgcodes/zotero-organizer.git
cd zotero-organizer
```

4.4 Run the setup wizard

```
python scripts/setup.py
```

The wizard prompts for each configuration value. Press Enter to accept the value shown in brackets, or type a replacement.

```
============================================================
  Zotero Organizer — first-time setup
============================================================

Press Enter to accept the value shown in brackets.

Project root (where collections/ will live)
  [/path/to/zotero-organizer]:

Numeric Zotero user ID  (zotero.org → Settings → Feeds/API)
  [required]: 1234567

Zotero API key with Read/Write access
  [required]:

Drive folder ID from the URL: drive.google.com/drive/folders/<ID>
  [required]: 1aBcDeFgHiJkLmNoPqRsTuVwXyZ

rclone remote name configured for Google Drive
  [gdrive]:

.env written → /path/to/zotero-organizer/.env
collections/ ready → /path/to/zotero-organizer/collections

--- Validating configuration ---
  ✓ rclone remote 'gdrive' found
  ✓ Zotero API credentials valid
  ✓ Google Drive folder accessible

============================================================
  Setup complete. Next step:

  python scripts/organize.py <pdf-folder> --collection <name>
============================================================
```

If any validation check fails, the wizard prints the specific error and the corrective step. Re-run `python scripts/setup.py` at any time to update a single value without re-entering the rest.

---

5. Organizing your first collection

Run the main script, pointing it at your PDF folder and giving the collection a name:

```
python scripts/organize.py /path/to/your/pdfs --collection "Plasmons"
```

The tool runs three stages in sequence. Each stage produces output files in a workspace directory (`collections/Plasmons/`) inside the project folder. **Your original PDF folder is never modified.**

5.1 Stage 1 — Scan

The scan stage walks your PDF folder recursively and attempts to extract a DOI for each file using a four-step fallback chain:

1. **PDF metadata** — checks the `/doi`, `/DOI`, and `/Subject` metadata fields.
2. **Page-1 text** — searches the first page for DOI and arXiv ID patterns.
3. **Semantic Scholar** — searches by the PDF's title (can be skipped with `--no-ss`).
4. **Fallback header** — stores the first 300 characters of page-1 text for manual resolution.

The scan also detects duplicate PDFs (by DOI). When duplicates are found, the most deeply nested copy is kept as canonical and the others are flagged.

The result is written to `collections/Plasmons/scan.json`. On subsequent runs, the scan is incremental — only new files are processed.

5.2 Stage 2 — Build collection structure

After the scan you are asked how to build the collection hierarchy:

```
Choose how to build the collection structure:
  [1] Scaffold — generate taxonomy.yaml, edit manually        <-- default
  [2] Auto     — use subfolder names directly, no editing
  [3] LLM/Claude — use taxonomy.yaml edited with Claude or another LLM
  Choice [1]:
```

Option 1 — Scaffold (recommended for first-time use)

Generates a `taxonomy.yaml` file pre-filled from your subfolder structure. The tool then exits cleanly:

```
taxonomy.yaml written → collections/Plasmons/taxonomy.yaml

Edit it in your preferred editor, then re-run:
  python scripts/organize.py /path/to/pdfs --collection Plasmons
```

Open `taxonomy.yaml`, review and adjust the collection structure and assignments (see Section 6), then re-run the command. The tool detects the existing taxonomy and skips Stage 1.

Option 2 — Auto

Uses your subfolder names as collection names directly. No editing step — the batch script is generated immediately and Stage 3 begins. Best suited for folders that are already organised the way you want them in Zotero.

Option 3 — LLM/Claude

Same as scaffold, but signals that you intend to have Claude or another LLM review and edit the taxonomy before proceeding. See Section 9.1 for the Claude workflow.

5.3 Stage 3 — Dry run and real run

Before making any changes, the tool always runs a dry run:

```
============================================================
  Stage 3 — Dry run validation
============================================================
=== DRY RUN — no Zotero or Drive changes will be made ===

  Collection  : Plasmons
  Items       : 22
  Flagged     : 3  (no DOI — will be skipped unless resolved)
  Duplicates  : 1  (skipped)

  [CREATE] Zotero collection: Plasmons
  [CREATE] Zotero collection: Plasmons > Quantum-Plasmonics
  [ITEM]   10.1103/PhysRevLett.90.057401 → Quantum-Plasmonics
  ...
=== DONE: 22/22 in 1s ===

Run for real? This will create Zotero items and upload to Drive. [y/N]:
```

Review the output, then type `y` to proceed or `N` to stop. If you type `N`, the generated `batch_run.py` is left in the workspace and can be run directly later:

```
python collections/Plasmons/batch_run.py
```

Progress is saved to `collections/Plasmons/state.json` after each paper. If the run is interrupted, re-running the same command (or `batch_run.py` directly) resumes from where it left off.

---

6. Working with taxonomy.yaml

`taxonomy.yaml` is the central configuration file for a collection. It has four top-level sections.

6.1 Collections

Defines the sub-collection hierarchy to create inside Zotero:

```yaml
base_collection: "Plasmons"

collections:
  - name: "Quantum-Plasmonics"
    parent: null                    # top-level under Plasmons
  - name: "Hydrodynamic-Modeling"
    parent: "Quantum-Plasmonics"    # nested under Quantum-Plasmonics
```

The same hierarchy is mirrored in Google Drive: `Plasmons/Quantum-Plasmonics/Hydrodynamic-Modeling/`

6.2 Assignments

Maps each PDF to a collection and a list of tags:

```yaml
assignments:
  "SubfolderA/paper.pdf":
    collection: "Quantum-Plasmonics"
    tags: ["review", "2024"]
  "SubfolderB/another.pdf":
    collection: "Hydrodynamic-Modeling"
    tags: []
```

Paths are relative to the PDF root folder you passed to `organize.py`.

6.3 Flagged papers

PDFs for which no DOI could be resolved automatically are placed in the `flagged` section with a suggested search query:

```yaml
flagged:
  - file: "SubfolderB/mystery.pdf"
    search_query: "Plasmon resonance gold nanoparticle ..."
    collection: "Quantum-Plasmonics"
    tags: []
```

To resolve a flagged paper: 1. Search Semantic Scholar (semanticscholar.org) using the `search_query` text. 2. Find the correct paper and copy its DOI. 3. Either add the DOI to `scan.json` under the paper's entry and move the item from `flagged` to `assignments`, or leave it in `flagged` to be skipped.

Flagged papers are skipped during the real run unless they are moved to `assignments` or `books`.

6.4 Books and theses

Non-journal items without DOIs (textbooks, theses, reports) are handled in the `books` section:

```yaml
books:
  - file: "Books/textbook.pdf"
    type: book            # book | thesis | document
    title: "Nanoplasmonics"
    authors:
      - first: "Stefan"
        last: "Maier"
    year: "2007"
    extra:
      publisher: "Springer"
    collection: "Books-Notes"
    tags: ["textbook"]
```

The `extra` field accepts any Zotero item field: `publisher`, `place`, `edition`, `university` (for theses), `institution` (for reports), etc.

---

7. Re-running when new PDFs are added

Simply run the same command again:

```
python scripts/organize.py /path/to/your/pdfs --collection "Plasmons"
```

The tool compares the current folder contents against the existing `scan.json` and reports what changed:

```
Found existing scan.json (2026-06-07, 22 files).
3 new PDFs detected, 0 removed.
  [1] Scan new files only (incremental)  <-- default
  [2] Full rescan
  [3] Skip scan
  Choice [1]:
```

Choose `[1]` to scan only the new files (recommended). The tool then writes a `taxonomy_patch.yaml` containing only the new papers:

```
3 new papers → collections/Plasmons/taxonomy_patch.yaml
Review and merge into taxonomy.yaml, then re-run this command.
```

Open `taxonomy_patch.yaml`, copy the new entries into the appropriate sections of `taxonomy.yaml`, and re-run. The batch script will skip papers already marked as done in `state.json`.

---

8. Drive conflict handling

On the first real run (when `state.json` does not yet exist), the tool checks whether the target Drive folder already contains files. If it does, you are asked how to proceed:

```
⚠  Drive folder 'Plasmons/' already contains 3 item(s):
     Quantum-Plasmonics/
     ...

  [1] Merge    – add new files, skip any that already exist  <-- default
  [2] Overwrite – replace existing files with local copies
  [3] New folder – upload to 'Plasmons_20260610/' instead
  [4] Abort
```

Choice	When to use
Merge	Continuing from a partial previous upload
Overwrite	Local files are newer and should replace the Drive copies
New folder	Starting fresh without disturbing the existing Drive structure
Abort	Stop and investigate before proceeding

This check is skipped on dry runs and when resuming an interrupted run.

---

9. Optional: Claude Desktop integration

9.1 Using Claude to suggest taxonomy

After Stage 1, the `taxonomy.yaml` scaffold contains all your papers grouped by subfolder. You can share this file with Claude (or any other LLM) and ask it to suggest better thematic groupings, collection names, and tags.

A useful prompt:

> Here is my taxonomy.yaml for a Zotero collection about plasmonics. Please reorganize the collections section into thematic sub-topics, reassign papers accordingly, and suggest two or three tags per paper based on the title.

Paste the contents of `taxonomy.yaml` into the conversation, apply Claude's suggestions, save the file, then re-run:

```
python scripts/organize.py /path/to/pdfs --collection Plasmons
```

Select option `[1]` (use existing taxonomy) at the Stage 2 prompt.

Note: pasting the taxonomy manually into a Claude conversation does not require Claude Desktop or the MCP server. It works in any Claude interface.

9.2 MCP server setup

`zotero_mcp.py` exposes the full workflow as callable tools inside Claude Desktop, enabling an interactive, per-PDF workflow where Claude can extract DOIs, create collections, upload files, and attach links on demand.

Additional requirements: - Claude Desktop installed on your machine. - The conda environment from Section 4.1 Option A (which includes the `mcp` package).

Step 1 — Find the conda Python path:

```
conda activate claudotero
which python
# e.g. /Users/you/miniconda3/envs/claudotero/bin/python
```

Note this path.

Step 2 — Add the MCP server to Claude Desktop's config:

Open the config file in a text editor: - macOS: `~/Library/Application Support/Claude/claude_desktop_config.json` - Linux: `~/.config/Claude/claude_desktop_config.json`

Add the following block (replace the placeholder paths and credentials):

```
{
  "mcpServers": {
    "zotero": {
      "command": "/path/to/conda/envs/claudotero/bin/python",
      "args": ["/path/to/zotero-organizer/zotero_mcp.py"],
      "env": {
        "ZOTERO_USER_ID": "your_numeric_user_id",
        "ZOTERO_API_KEY": "your_api_key",
```

```
        "GOOGLE_DRIVE_FOLDER_ID": "your_folder_id",
        "RCLONE_REMOTE": "gdrive"
      }
    }
  }
}
```

If the config file already has an `mcpServers` block, add the `"zotero"` entry inside the existing object.

Step 3 — Restart Claude Desktop.

The Zotero tools will appear as callable functions in your Claude conversation.

Available tools:

Tool	Description
`extract_doi_from_local_pdf`	Extract DOI from a local PDF file
`create_collection`	Create a Zotero collection (with optional parent)
`add_item_by_doi`	Resolve DOI via Semantic Scholar and create a Zotero item
`upload_pdf_to_drive`	Upload a PDF to Google Drive via rclone
`add_url_attachment`	Attach a URL (e.g. a Drive link) to a Zotero item
`get_drive_folder_id`	Return the configured Google Drive folder ID
`get_collection_structure`	List the sub-collections and items of a collection
`add_tags_to_item`	Add tags to an existing Zotero item

---

10. Reference: command flags

All flags are optional. Guided prompts fill in anything not specified.

```
python scripts/organize.py [folder] [--collection NAME] [options]
```

Flag	Effect
`--mode scaffold`	Generate `taxonomy.yaml` and exit
`--mode auto`	Use subfolder names directly, skip taxonomy editing
`--mode taxonomy`	Use existing `taxonomy.yaml` to generate the batch script
`--no-ss`	Skip Semantic Scholar DOI search (faster, offline-safe)
`--full-scan`	Force a complete rescan even if `scan.json` already exists
`--skip-scan`	Skip Stage 1 entirely and use the existing `scan.json`
`--dry-run`	Stop after dry-run validation without running for real
`--workspace <path>`	Override the default workspace location

To run the individual stage scripts directly, see the **Advanced** section of the README.

---

11. Reference: workspace layout

All generated files are written to `collections/<name>/` inside the project directory. Your PDF folder is never modified.

```
zotero-organizer/
  collections/
    Plasmons/
      scan.json            — Stage 1 output; updated incrementally on re-runs
      taxonomy.yaml        — Stage 2 scaffold; edit to customise structure
      taxonomy_patch.yaml  — new files found on re-scan; merge into taxonomy.yaml
      batch_run.py         — generated self-contained runner script
      state.json           — per-paper progress; delete to start a fresh run
```

By default `collections/` is excluded from git (`.gitignore`). To track taxonomy files across machines, remove the `collections/` line from `.gitignore`.

---

12. Troubleshooting

`rclone remote 'gdrive' not configured` Run `rclone config` and create a remote named `gdrive` (or whatever name you chose). See Section 4.2.

`Zotero API check failed: 403 Forbidden` Your API key does not have Write access. Delete the key at zotero.org → Settings → Feeds/API and create a new one with Read/Write ticked.

`Drive folder check failed` The folder ID may be wrong, or the Google account authenticated in rclone may not have access to that folder. Re-run `rclone ls gdrive: --drive-root-folder-id=<ID>` to diagnose.

DOI extraction returns many `unresolved` papers PDFs without machine-readable text (scanned images) cannot be parsed by the text regex. For these, add the DOI manually to `scan.json` or move them to the `books` section of `taxonomy.yaml`. Adding `--no-ss` speeds up the scan when offline but will reduce DOI resolution for papers that require the Semantic Scholar fallback.

`ModuleNotFoundError: No module named 'pyzotero'` The Python dependencies are not installed in the active environment. Run `pip install -r requirements.txt` or activate the conda environment (`conda activate claudotero`).

Interrupted run does not resume Check that `collections/<name>/state.json` exists. If it is empty or corrupt, delete it — the run will restart from the beginning. Papers already in Zotero will be detected as duplicates and skipped automatically.

Windows: `rclone` not found in WSL2 rclone must be installed inside WSL2, not only on the Windows side. Run `curl https://rclone.org/install.sh | sudo bash` inside the WSL2 terminal.